

Date and Time

Python provides the `datetime` module to work with dates and times. This module includes various classes and methods to handle date and time operations efficiently.

1. Importing the `datetime` Module

Before using date and time functionalities, you need to import the `datetime` module:

```
Python
import datetime
```

2. Working with Dates

The `date` class from the `datetime` module is used to work with dates.

2.1 Getting the Current Date

```
Python
from datetime import date

today = date.today()
print("Today's date:", today)
```

2.2 Creating a Specific Date

```
Python
specific_date = date(2024, 2, 16)
print("Specific date:", specific_date)
```

2.3 Extracting Year, Month, and Day

Python

```
print("Year:", today.year)
print("Month:", today.month)
print("Day:", today.day)
```

2.4 Finding the Day of the Week

Python

```
print("Weekday (0=Monday, 6=Sunday):", today.weekday()) #
Monday is 0
print("ISO Weekday (1=Monday, 7=Sunday):", today.isoweekday())
# Monday is 1
```

3. Working with Time

The `time` class from the `datetime` module is used to work with time.

3.1 Creating a Specific Time

Python

```
from datetime import time

specific_time = time(14, 30, 15) # 14:30:15 (2:30:15 PM)
print("Specific time:", specific_time)
```

3.2 Extracting Hours, Minutes, and Seconds

Python

```
print("Hour:", specific_time.hour)
print("Minute:", specific_time.minute)
print("Second:", specific_time.second)
```

4. Working with Date and Time Together

The `datetime` class from the `datetime` module handles both date and time.

4.1 Getting the Current Date and Time

```
Python
from datetime import datetime

now = datetime.now()
print("Current date and time:", now)
```

4.2 Creating a Specific Date and Time

```
Python
specific_datetime = datetime(2024, 2, 16, 14, 30, 15)
print("Specific date and time:", specific_datetime)
```

4.3 Extracting Date and Time Components

```
Python
print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)
print("Hour:", now.hour)
print("Minute:", now.minute)
print("Second:", now.second)
```

5. Formatting Dates and Times

Use the `strftime()` method to format date and time into readable strings.

5.1 Formatting Examples

```
Python
formatted_date = now.strftime("%Y-%m-%d")
formatted_time = now.strftime("%H:%M:%S")
formatted_datetime = now.strftime("%d-%b-%Y %I:%M %p")
```

```
print("Formatted Date:", formatted_date)
print("Formatted Time:", formatted_time)
print("Formatted Date and Time:", formatted_datetime)
```

5.2 Common Formatting Codes

Code	Description	Example
%Y	Full Year	2024
%m	Month (01-12)	02
%d	Day (01-31)	16
%H	Hour (00-23)	14
%I	Hour (01-12)	02
%M	Minutes (00-59)	30
%S	Seconds (00-59)	15
%p	AM/PM	PM
%A	Full Weekday Name	Friday
%B	Full Month Name	February

6. Parsing Strings into Dates

Use `strptime()` to convert a string into a `datetime` object.

```
Python
date_string = "16-02-2024 14:30"
parsed_date = datetime.strptime(date_string, "%d-%m-%Y %H:%M")
print("Parsed Date and Time:", parsed_date)
```

7. Date and Time Arithmetic

You can perform arithmetic operations using `timedelta`.

Python

```
from datetime import timedelta

# Adding 7 days
future_date = today + timedelta(days=7)
print("Date after 7 days:", future_date)

# Subtracting 3 days
past_date = today - timedelta(days=3)
print("Date 3 days ago:", past_date)

# Adding 2 hours
future_time = now + timedelta(hours=2)
print("Time after 2 hours:", future_time)
```